PRACTICAL LABORATORY MANUAL

Smart Automatic Dustbin
Using Arduino UNO, Ultrasonic Sensor & Servo Motor

Proteus ISIS Simulation and Hardware Implementation

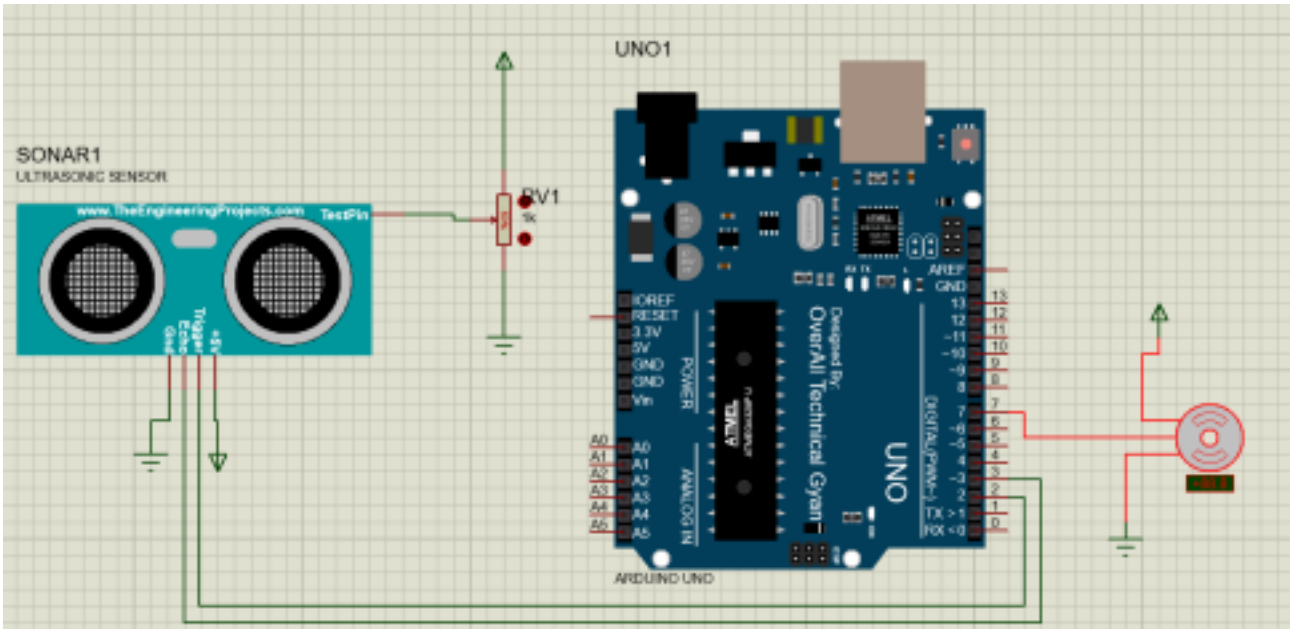


Figure 1. User-provided final Proteus circuit schematic.

Important note about the supplied schematic

The code and connection table in this manual are mapped to the visible wiring in the supplied screenshot. Tracing the wires in the image indicates: SONAR TRIG → Arduino D3, SONAR ECHO → Arduino D2, servo signal → Arduino D6. The SONAR model's TestPin is connected to the RV1 potentiometer wiper; the potentiometer's outer terminals are connected to the positive supply and ground.

Before compiling: verify these three pin labels against your Proteus schematic. If you intentionally rewire any signal to another Arduino pin, change the corresponding constant in the code.

Model-specific caution: SONAR1 is a Proteus ultrasonic-sensor model from TheEngineeringProjects, not necessarily the same model as a standard HC-SR04 VSM component. Its TestPin/potentiometer behavior depends on the installed model. A symbol alone does not guarantee a working simulation model.

1. Experiment information

Experiment title: Design and simulation of a smart automatic dustbin using Arduino UNO, an ultrasonic sensor model (SONAR1), and a servo motor.

Objectives

• Understand the operating principle of an ultrasonic distance sensor.
• Interface an ultrasonic sensor model with Arduino UNO using digital input/output pins. •
Control a servo motor using the Arduino Servo library.
• Use distance-based decision logic to open and automatically close a dustbin lid. •
Build and test the circuit in Proteus ISIS and interpret the simulation results.

2. Required components and software

Item	Quantity	Purpose / notes
Arduino UNO R3	1	Main controller; ATmega328P board model.
SONAR1 ultrasonic sensor model	1	Proteus sensor model shown in the supplied schematic.
RV1 potentiometer, 1 kΩ	1	Connected to SONAR1 TestPin to vary the model input.
Servo motor model	1	Moves the dustbin lid; signal is wired to D6 in the screenshot.
5 V supply and GND terminals	As required	Common reference and supply for simulated modules.

Proteus ISIS	1	Schematic capture and simulation.
Arduino IDE	1	Compile sketch and export HEX file.

For a physical build, use an appropriate regulated 5 V supply for the servo if required by its current demand. Join the external supply ground to Arduino ground. Do not assume the Arduino 5 V pin can safely supply a loaded servo.

3. Circuit connections

3.1 Signal and power connections

From component	Pin / terminal	Connect to	Purpose
SONAR1	TRIG	Arduino D3	Arduino sends the trigger pulse.
SONAR1	ECHO	Arduino D2	Arduino measures the returned pulse.
SONAR1	+5V	5 V supply	Sensor power.
SONAR1	GND	Common GND	Ground reference.
SONAR1	TestPin	RV1 wiper / middle terminal	Model-specific distance control input.
RV1 (1 kΩ)	One outer terminal	+5 V	Potentiometer supply end.
RV1 (1 kΩ)	Other outer terminal	GND	Potentiometer ground end.
Servo	Signal	Arduino D6	Servo position command.
Servo	VCC / positive	+5 V	Servo supply in the simulation.
Servo	GND / negative	Common GND	Shared ground reference.

3.2 Pin mapping summary

Arduino UNO pin	Connected to
D2	SONAR1 ECHO
D3	SONAR1 TRIG
D6	Servo signal
5V	SONAR1 +5V and servo VCC, as represented in the simulation
GND	SONAR1 GND and servo GND

Do not connect the potentiometer wiper to A0 for this particular schematic unless you redesign the circuit and update the program. In the supplied design, the wiper goes to the SONAR1 TestPin.

3.3 How to wire in Proteus

1. Place Arduino UNO R3, SONAR1, RV1 (1 kΩ), servo motor, +5 V power terminal, and GND terminals.
2. Connect SONAR1 TRIG to Arduino D3.
3. Connect SONAR1 ECHO to Arduino D2.
4. Connect SONAR1 +5V and GND to the corresponding power nets.

5. Connect the RV1 wiper (middle terminal) to SONAR1 TestPin. Connect the two outer terminals to +5 V and GND, respectively.
6. Connect the servo signal wire to Arduino D6, servo positive to +5 V, and servo ground to common GND.
7. Check that all ground connections share the same electrical ground net.

4. Working principle

The Arduino repeatedly triggers the ultrasonic sensor model and measures the duration of the ECHO pulse. The pulse duration is converted to an estimated distance. When the measured distance is positive and no greater than the configured threshold (20 cm), the Arduino commands the servo to move to the open position (90°). Each valid detection refreshes a timer. Once no new detection has occurred for 1.5 seconds, the servo is commanded back to the closed position (0°).

4.1 Distance equation

Distance (cm) = duration (µs) × 0.0343 ÷ 2

The approximate speed of sound used by the code is 0.0343 cm/µs. The division by two is necessary because the measured echo pulse represents the sound wave's round trip: sensor → object → sensor.

4.2 Role of the potentiometer

RV1 changes the voltage applied to the SONAR1 TestPin. In this particular Proteus model, that input is intended to control the simulated sensor condition/distance. The exact voltage-to-distance relationship is model-specific; do not assume that a particular knob percentage always equals a particular distance

unless you have verified the model documentation or calibrated it during simulation.

5. Complete Arduino program for the supplied circuit

This sketch uses the pin mapping traced from the supplied schematic: TRIG = D3, ECHO = D2, and servo signal = D6.

```
#include <Servo.h>

// Create a Servo object
Servo myservo;

// Pin connections from the supplied Proteus schematic
const byte trigPin = 3; // SONAR1 TRIG -> Arduino D3
const byte echoPin = 2; // SONAR1 ECHO -> Arduino D2
const byte servoPin = 6; // Servo signal -> Arduino D6

// Distance measurement variables
unsigned long duration;
float distanceCm;

// Dustbin movement settings
const byte openAngle = 90;
const byte closeAngle = 0;
const float detectionDistanceCm = 20.0;
```



```
// Automatic closing timer
unsigned long lastDetectionTime = 0;
const unsigned long closeDelayMs = 1500;

// Current lid state
bool isOpen = false;

void setup()
{
 pinMode(trigPin, OUTPUT);
 pinMode(echoPin, INPUT);

 myservo.attach(servoPin);
 myservo.write(closeAngle); // Start with the lid closed

 Serial.begin(9600);
 delay(500);
 }

void loop()
{
 // Send a clean 10 microsecond trigger pulse
digitalWrite(trigPin, LOW);
```

```
 delayMicroseconds(2);

 digitalWrite(trigPin, HIGH);
 delayMicroseconds(10);
 digitalWrite(trigPin, LOW);

 // Measure the HIGH pulse on ECHO; timeout = 30 ms
duration = pulseIn(echoPin, HIGH, 30000UL);

 // Convert round-trip time to one-way distance
if (duration == 0)
 {
 distanceCm = -1.0; // No valid echo before timeout  }
 else
 {
 distanceCm = (duration * 0.0343) / 2.0;
 }

 // Print the measurement for debugging
 Serial.print("Distance: ");
 if (distanceCm < 0)
 {
 Serial.println("No echo");
 }
 else
```



```
 {
 Serial.print(distanceCm);
 Serial.println(" cm");
 }

 // If an object is within the detection range, open/keep open
 if (distanceCm > 0 && distanceCm <= detectionDistanceCm)
 {
 myservo.write(openAngle);
 lastDetectionTime = millis();
 isOpen = true;
 Serial.println("DUSTBIN OPEN");
 }

 // Close only after the last valid detection has expired
 if (isOpen && (millis() - lastDetectionTime >= closeDelayMs))
 {
 myservo.write(closeAngle);
 isOpen = false;
 Serial.println("DUSTBIN CLOSED");
 }

 delay(100);
```

```
}
```

6. Program explanation: line by line (English and Bangla)

#include <Servo.h>

: Includes the Arduino Servo library, which provides functions for commanding a servo motor.

Servo myservo;

: Creates a Servo object named myservo. The object is used to attach and command the servo. **const byte trigPin = 3;**

: Defines Arduino D3 as the trigger output connected to SONAR1 TRIG.

const byte echoPin = 2;

: Defines Arduino D2 as the echo input connected to SONAR1 ECHO.

const byte servoPin = 6;

: Defines Arduino D6 as the servo control-signal pin, matching the supplied schematic. **unsigned long duration;**

: Stores the measured ECHO pulse duration in microseconds. unsigned long can hold a large non negative integer.

float distanceCm;

: Stores the calculated distance in centimeters, including decimal values.

const byte openAngle = 90; / closeAngle = 0;

: Defines the servo positions for opening and closing. These angles may need mechanical calibration. **const float detectionDistanceCm = 20.0;**

: Sets the object-detection threshold to 20 cm.

lastDetectionTime and closeDelayMs

: Store the time of the most recent detection and the 1500 ms waiting period before closing. **bool isOpen = false;**

: Tracks whether the program currently considers the lid open. Initially it is closed. **void setup()**

: Runs once after the Arduino starts or resets.

pinMode(trigPin, OUTPUT);

: Configures D3 as an output so Arduino can send a trigger pulse.

pinMode(echoPin, INPUT);

: Configures D2 as an input so Arduino can read the sensor's echo pulse.

myservo.attach(servoPin);

: Associates the Servo library object with D6.

myservo.write(closeAngle);

: Commands the servo to the closed position at startup.

Serial.begin(9600);

: Starts serial communication at 9600 bits per second for diagnostic output.

digitalWrite(trigPin, LOW); delayMicroseconds(2);

: Holds TRIG LOW briefly to establish a clean starting state.

digitalWrite(trigPin, HIGH); delayMicroseconds(10);

: Creates the 10 microsecond HIGH trigger pulse required by common ultrasonic modules. **digitalWrite(trigPin, LOW);**

: Ends the trigger pulse.

pulseIn(echoPin, HIGH, 30000UL);

: Measures how long D2 remains HIGH. The third argument sets a 30,000 microsecond timeout. **if (duration == 0)** A zero result means no HIGH echo pulse was received before the timeout.

distanceCm = (duration * 0.0343) / 2.0: Converts the round-trip echo time into one-way distance in centimeters using the approximate speed of sound.

Serial.print() and Serial.println() Print the measured distance or 'No echo' so the user can inspect the sensor behavior.

if (distanceCm > 0 && distanceCm <= detectionDistanceCm): Accepts only a valid positive distance that is at or below 20 cm.

myservo.write(openAngle); Commands the servo to move to 90 degrees, opening the lid.

lastDetectionTime = millis(); Records the current Arduino uptime in milliseconds. Each new detection refreshes the timer.

if (isOpen && millis() - lastDetectionTime >= closeDelayMs) Closes the lid only when it is open and at least 1500 ms have passed since the last detection.

myservo.write(closeAngle); Commands the servo to return to zero degrees.

delay(100); Adds a short 100 ms pause between measurement cycles.7. Proteus simulation procedure

8. Open Proteus ISIS and create a new project. Choose a schematic-only project if PCB design is not required.
9. Press P (Pick Devices) and search for Arduino UNO R3. Place it on the schematic. 10. Place the SONAR1 ultrasonic sensor model used in the supplied schematic. Search the installed library for SONAR/ultrasonic sensor; model names differ between libraries.
11. Place RV1 and set its resistance to 1 kΩ. Connect its wiper to SONAR1 TestPin and its outer terminals to +5 V and GND, matching the supplied schematic.
12. Place a servo motor model that supports the control signal used by the Arduino simulation. 13. Wire TRIG to D3, ECHO to D2, and servo signal to D6. Complete the power and common-ground connections.
14. Open Arduino IDE, paste the complete program, and select Tools → Board → Arduino AVR Boards → Arduino Uno.
15. Compile the sketch. Use Sketch → Export Compiled Binary. Locate the generated .ino.hex file in the sketch folder.
16. In Proteus, double-click the Arduino UNO. Set the Program File to the regular .ino.hex file (not the with_bootloader file, unless your specific simulation setup requires otherwise).
17. Set the Arduino model clock to 16 MHz if the component exposes a clock-frequency property.

18. Run the simulation. Adjust RV1 gradually and observe whether the SONAR1 model generates an ECHO pulse and whether the servo responds.
19. Open Arduino Serial Monitor only when testing on a real Arduino or when your Proteus setup provides a virtual terminal/serial interface.

7.1 How to export the HEX file

Arduino IDE:

1. Tools -> Board -> Arduino AVR Boards -> Arduino Uno
2. Sketch -> Export Compiled Binary
3. Wait for compilation to finish
4. Open the sketch folder
5. Select: ProjectName.ino.hex

The normal .ino.hex is generally the appropriate file to load into the Proteus Arduino model. The .with_bootloader.hex variant includes bootloader content and is usually not needed for this simulation.

7.2 Testing RV1 / SONAR1

Rotate RV1 while the simulation is running. Observe the SONAR1 model and the ECHO signal. The sensor model should produce a valid ECHO pulse corresponding to its simulated distance. When the resulting distance is 20 cm or less, the servo should move to the open angle. If the model does not generate an ECHO pulse, the code will report 'No echo' and will not open the lid.

Important limitation: the potentiometer position is not a universal distance calibration. The voltage-to distance mapping depends on the exact SONAR1 Proteus model/library version. If the installed model has a documented TestPin range, use that documentation to set and interpret RV1.

8. Expected results and observation table

Test	Distance condition	Expected output
1	Valid distance greater than 20 cm	Servo remains at 0°; lid closed.

2	Valid distance equal to or below 20 cm	Servo moves to 90°; lid opens.
3	Object remains within 20 cm	Lid remains open; detection timer refreshes.
4	Object moves outside range / no valid detection	After 1.5 s from last detection, servo returns to 0°.
5	No echo received	Distance is treated as invalid; no new opening command.

Record the actual RV1 position, sensor output, and servo response during your own simulation. Do not report assumed distances unless they are verified from the model output.

9. Troubleshooting

Problem	Likely cause	What to check
Servo does not move	Wrong servo pin or incompatible model	Verify servo signal is D6; check servo model supports servo pulses.
Serial output says No echo	No valid ECHO pulse	Check ECHO-to-D2 wiring, sensor model, TestPin, and common GND.
Sensor does not react to RV1	Model's TestPin behavior differs	Check the installed SONAR1 library documentation and pin names.
Arduino does not run	HEX file not loaded or wrong board model	Check Program File, compilation result, and 16 MHz clock.
Servo opens at unexpected distance	Model calibration differs	Verify actual distance output and calibrate the TestPin voltage.
Servo jitters or resets in hardware	Insufficient supply current or noisy wiring	Use a suitable regulated servo supply and common ground.
Lid angle is incorrect	Mechanical orientation differs	Adjust openAngle and closeAngle carefully.

10. Safety and implementation notes

• In a real build, do not power a loaded servo directly from an Arduino I/O pin. • Use a regulated supply suitable for the servo's voltage and current requirements; connect grounds together.

• Do not force the servo arm against a jammed lid or mechanical stop.

• Keep wires away from the moving lid mechanism.

• The 0° and 90° positions are starting values; calibrate them for the physical linkage. • The Proteus SONAR1 TestPin behavior is model-specific. Validate the simulation model before interpreting RV1 position as an actual distance.

11. Viva voce questions and answers

Q. What is Arduino UNO?

Answer: An open-source microcontroller development board based on the ATmega328P.

Q. What is the purpose of an ultrasonic sensor?

Answer: It estimates distance by transmitting sound waves and measuring the returning echo. **Q. What does TRIG do?**

Answer: It receives a trigger pulse from the controller to start a measurement.

Q. What does ECHO do?

Answer: It provides a pulse whose duration represents the ultrasonic round-trip travel time. **Q. Why is the distance formula divided by two?**

Answer: Because the sound travels to the object and back.

Q. What is a servo motor?

Answer: A motor with position control, commonly used to move to a commanded angular position. **Q. Why is the Servo library included?**

Answer: It provides convenient functions to generate servo control pulses.

Q. What does pulseIn() return?

Answer: The duration of a specified HIGH or LOW pulse in microseconds, or zero if it times out. **Q. What is millis()?**

Answer: A function that returns the number of milliseconds since the Arduino program started. **Q. Why use millis() for the closing timer?**

Answer: It allows the program to measure elapsed time without blocking for the whole waiting period.

Q. What is the role of RV1 in this schematic?

Answer: It varies the voltage applied to SONAR1 TestPin; its exact simulation meaning depends on the installed model.

Q. Why must grounds be common?

Answer: The connected circuits need a shared voltage reference for their signals.

12. Conclusion

In this experiment, an Arduino UNO is interfaced with an ultrasonic sensor model and a servo motor to implement a smart automatic dustbin. The controller measures the echo pulse, calculates distance, compares it with a 20 cm threshold, and controls the servo to open and close the lid. The Proteus simulation demonstrates the interaction between sensor input, embedded decision logic, and actuator output. Successful simulation depends on using a functional SONAR1 model whose TestPin and ECHO behavior match the installed library.